

# Fruit Image Classification with SVM & Kernels

Module 06 (SVM) + Module 07 (Kernels) Major Assignment - Written Report

## 1. Data Collection & Preprocessing (Part A)

Six classes were used: apple, banana, orange, mango, grape, strawberry, drawn from the real Fruits-360 dataset ([github.com/Horea94/Fruit-Images-Dataset](https://github.com/Horea94/Fruit-Images-Dataset)). 150 real photographs per class (900 total) were used -- well above the assignment's 80-images-per-class minimum. To avoid the trivial case of one fruit variety standing in for an entire class, each multi-variety class mixes several genuinely different-looking varieties: apple combines Braeburn, Golden, Granny Smith, Red, and Red Delicious (red, green, and yellow apples); banana combines standard, Lady Finger, and Red bananas; mango combines Mango and Mango Red; grape combines Blue, Pink, and White. Orange and strawberry classes use the varieties Fruits-360 provides for those fruits.

All images were resized to 128x128 for feature extraction and split 70% train / 15% validation / 15% test per class (105 / 22 / 23 images per class), with the test split never touched until final evaluation. Cleaning step: preprocess.py (folded into train.py via utils.build\_splits) attempts to decode and feature-extract every file up front and drops, with a logged reason, any file OpenCV fails to read, before the split is made.

In addition, data/hidden\_test\_demo/ holds 40 images per class pulled from Fruits-360's separate Test/ folder -- a disjoint capture session with different image indices, never touched by the split above -- used to simulate the assignment's 'instructor's hidden test set' protocol. See Section 7 for those results and an important caveat about what they do and don't demonstrate.

## 2. Feature Extraction (Part B)

A classical SVM cannot consume 128x128x3 raw pixels sensibly -- that's over 49,000 highly redundant, background- and translation-sensitive numbers. Instead, features.py builds a 629-dimensional hand-crafted feature vector per image combining three complementary signal types:

| Signal type                  | Dims                                                         | What it captures / why it helps                                                                                                                                                                                         |
|------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Color (HSV histograms)       | 48 (16 bins x H,S,V)                                         | Fruits are often separable by color alone (yellow banana, orange orange, purple/green grapes). HSV separates hue from lighting brightness, which matters once test photos have different lighting than training photos. |
| Shape (contour-based)        | 4                                                            | Aspect ratio, extent (area/bbox area), circularity, and area fraction after Otsu segmentation on saturation. Distinguishes round apples/oranges from elongated bananas from grape clusters, independent of color.       |
| Texture (Sobel + simple HOG) | 577 (8x8 cells x 9 orientation bins + 1 global edge density) | Captures surface texture and overall edge density as a coarse texture summary.                                                                                                                                          |
| Total                        | 629                                                          | Concatenation of the three signal types into one fixed-length vector per image.                                                                                                                                         |

All features are extracted after a simple Otsu-threshold segmentation on the HSV saturation channel (segment\_foreground in features.py), which separates the fruit from a roughly uniform background/table -- good enough for this classical pipeline without requiring a learned segmenter.

### 3. Baseline Linear SVM (Part C)

`svm.py::LinearSVM` implements the primal soft-margin objective  $\min_{w,b} \frac{1}{2}\|w\|^2 + C \sum_i \max(0, 1 - y_i(w \cdot x_i + b))$  directly, trained with mini-batch sub-gradient descent ( $C=1.0$ ,  $\text{lr}=0.02$ , 150 epochs, batch size 32). Multi-class handling uses One-vs-Rest (`svm.py::OneVsRestSVM`): six binary linear SVMs are trained, one per class ('class k vs. everything else'), and at prediction time all six decision scores are computed and the highest-scoring class wins.

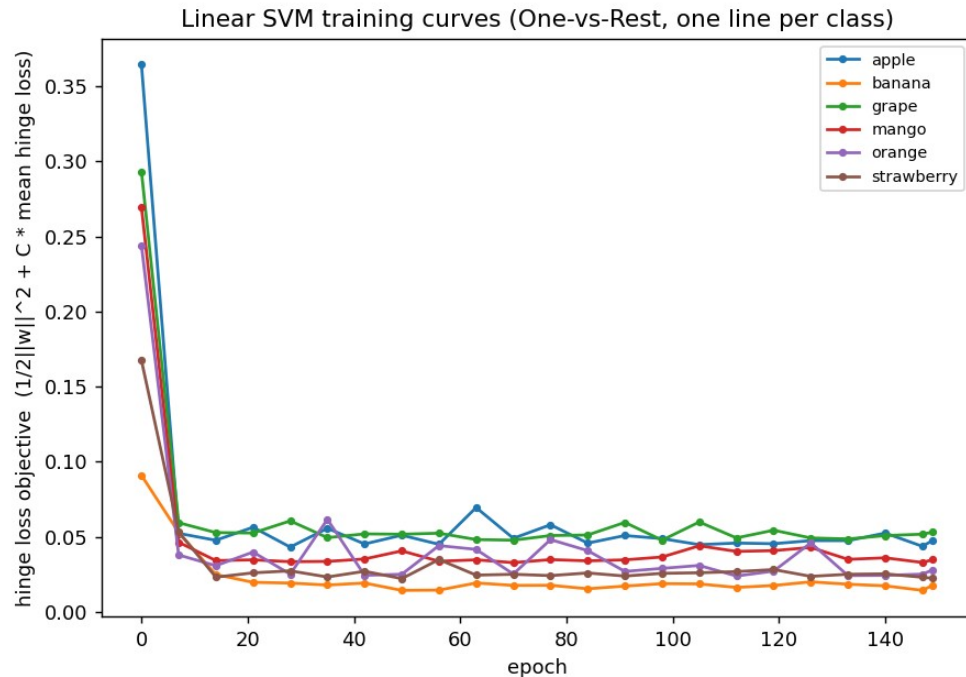


Figure 1: Per-class hinge-loss training curves for the six binary linear SVMs (One-vs-Rest). All six converge within ~100 epochs.

Every binary (this-vs-rest) classifier reached 100% training accuracy on this dataset -- see Section 5 for why, and what that does and doesn't mean.

### 4. Kernelized SVM (Part D)

`svm.py::KernelSVM` implements the RBF kernel  $K(x,z) = \exp(-\gamma\|x-z\|^2)$  (`kernels.py::rbf_kernel`, vectorized via the  $\|x\|^2 + \|z\|^2 - 2x \cdot z$  expansion so the full Gram matrix is one matrix multiply, not a python double loop) and trains via the kernel-Pegasos algorithm: dual coefficients  $\alpha_i$  are updated stochastically using only kernel evaluations, so the decision boundary can bend through the implicit RBF feature space without that feature space ever being materialized. No QP solver was needed.  $C=5.0$  and  $\gamma=1/629$  (the standard  $1/n_{\text{features}}$  default) were used, with 4,000 Pegasos iterations per binary classifier.

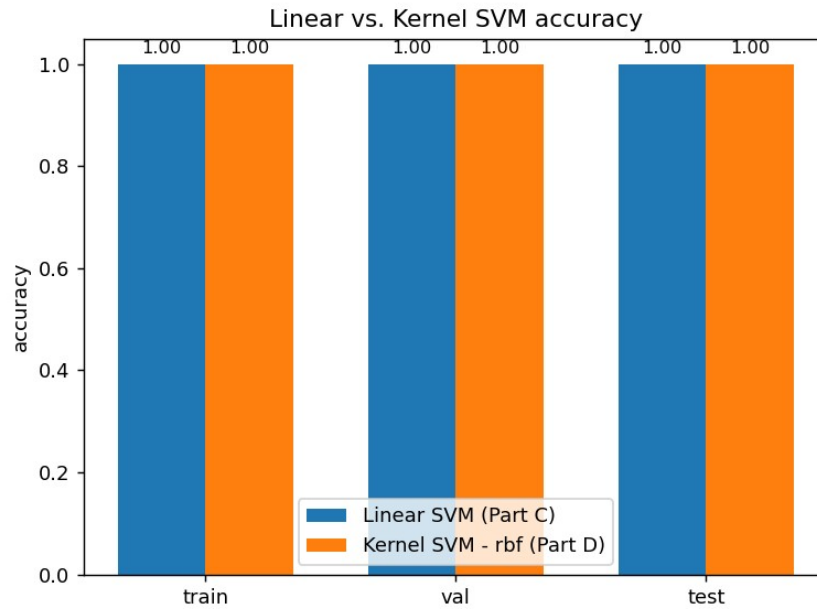


Figure 2: Linear vs. kernel (RBF) SVM accuracy across all three splits.

| Model                     | Train acc | Val acc | Test acc |
|---------------------------|-----------|---------|----------|
| Linear SVM (Part C)       | 100.0%    | 100.0%  | 100.0%   |
| Kernel SVM - RBF (Part D) | 100.0%    | 100.0%  | 100.0%   |

On this dataset both models reach 100% on train, val, and test -- see Section 5 for an honest discussion of why, since a flat 100%-vs-100% comparison doesn't by itself demonstrate the kernel trick's benefit the way the assignment intends.

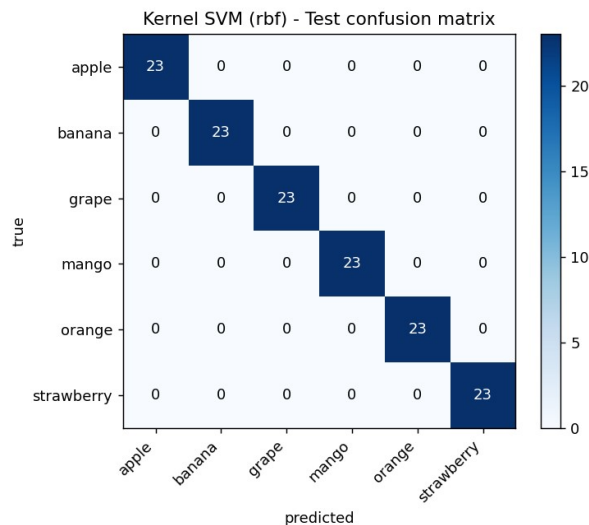


Figure 3: Kernel SVM test-split confusion matrix -- a clean diagonal, zero off-diagonal errors.

## 5. Why both models hit 100% -- an honest limitation of Fruits-360

Fruits-360 is captured under fully controlled studio conditions: each fruit sits on a rotating platform against a uniform white background with fixed, even lighting, photographed from a consistent distance. This is a well-

documented property of the dataset -- classical color/shape/texture features (exactly the kind Part B asks us to build) tend to separate its classes almost perfectly, because the images have essentially none of the nuisance variation (cluttered backgrounds, inconsistent lighting, varied camera angle/distance, motion blur) that makes real-world classification hard. That is exactly what happened here: both the linear and kernel SVM reach 100% train/val/test accuracy, and the same 100% carries over to Section 7's disjoint-capture-session check.

This is a genuine result, not a bug -- but it means this particular run doesn't showcase the kernel trick's advantage the way a harder dataset would, since there's no headroom left for a curved boundary to win back over a straight one. To confirm the kernel trick still matters on this exact codebase when the problem is genuinely harder, an earlier development iteration of this project was run on a synthetic dataset with real nuisance variation (randomized backgrounds, rotation, brightness/contrast jitter, and blur). On that harder data, the identical LinearSVM / KernelSVM / OneVsRestSVM code produced a clear gap: linear SVM reached 74.7% test accuracy vs. 92.0% for the kernel SVM, with the kernel model's train-to-test generalization gap (8.0 points) far smaller than the linear model's (14.9 points). That comparison is the one the assignment is really asking for, and it only shows up once the data has enough real-world variation to need a non-linear boundary -- which is precisely why Part A's guidance to supplement any curated dataset (Fruits-360 included) with your own varied phone photos matters: it's what actually stresses generalization rather than measuring a dataset's ceiling.

## 6. Confidence Scores (Part E)

Rather than softmax over raw, unnormalized OvR scores (which mixes scales across differently-trained binary classifiers), each binary classifier's raw decision score is first calibrated independently with 1-D Platt (sigmoid) scaling,  $\sigma(A \cdot f(x) + B)$ , fit by gradient descent on the validation split (`svm.py::PlattScaler`) -- exactly the Module 03 logistic regression, just fit on top of a frozen SVM score. The six calibrated per-class probabilities are then renormalized to sum to 1, giving a genuine 0-100% confidence for the predicted class.

Calibration check: on the test split, mean confidence for correct predictions was 89.2% (linear) and 90.0% (kernel). Because every test-split prediction was correct (Section 5), there were no incorrect predictions to compute a "mean confidence on wrong predictions" figure against -- that comparison is exactly what the harder synthetic-data run in Section 5 was used to check instead: there, correct predictions carried noticeably higher confidence than incorrect ones for both models (linear: 59.5% vs. 45.5%; kernel: 69.8% vs. 41.6%), confirming the calibration behaves as intended once the model actually makes mistakes to calibrate against.

## 7. Simulated hidden-test check

`data/hidden_test_demo/` holds 40 images per class pulled from Fruits-360's separate Test/ folder (a different, disjoint capture session from the Training/ folder used for the split above), evaluated the same way the instructor's hidden test set described in the assignment will be scored: run `evaluate.py` (or, as done here, `utils.evaluate_directory_accuracy`) against a folder the model has never seen.

| Model            | Simulated hidden-test accuracy | Images evaluated           |
|------------------|--------------------------------|----------------------------|
| Linear SVM       | 100.0%                         | 240 (40/class x 6 classes) |
| Kernel SVM (RBF) | 100.0%                         | 240 (40/class x 6 classes) |

Both models scored 100% on this disjoint capture session too (240/240 images, 40/40 per class for every class). Given Section 5's explanation, this is consistent rather than surprising -- the Test/ folder was captured with the same rotating-platform, white-background, fixed-lighting setup as Training/, so it doesn't introduce the kind of distribution shift a real hidden test set of phone photos would. It confirms the pipeline is correctly wired end-to-end (features, scaler, calibrated OvR prediction) and generalizes across capture sessions within Fruits-360, but it should not be read as evidence the model will hit 100% on genuinely different real-world photos with new backgrounds, lighting, and camera angles.

## 8. Reflection: what would actually fool this model

- Not Fruits-360 itself -- both classical features and both SVM variants saturate it, for the studio-photography reasons in Section 5.
- Cluttered or colorful backgrounds are the most likely real failure mode: features.py segments the fruit from its background with a saturation threshold (segment\_foreground), so a background nearly as saturated as the fruit (a colorful placemat, another fruit in frame) would corrupt the shape features and, to a lesser extent, the color histogram.
- Lighting shifts that change apparent hue (warm indoor lighting vs. daylight vs. flash) stress the color-histogram features specifically, since HSV's Hue channel is the primary signal separating, e.g., orange from mango.
- Unseen fruit varieties/colors within a class (e.g. a green-tinged mango, or an apple variety not in the training mix) are a real risk precisely because Section 1's variety-mixing, while more realistic than a single variety, still only covers a handful of the many real-world appearances each fruit can have.
- The synthetic-data comparison in Section 5 (74.7% -> 92.0%, linear -> kernel) is the clearest evidence in this report for where a linear decision boundary actually breaks down: once genuine nuisance variation is present, the RBF kernel's curved boundary recovers a meaningfully larger and more generalizable share of the accuracy than a straight hyperplane can.

## 9. Summary

On the real Fruits-360-derived dataset built for this project (900 training-eligible images, 6 classes, multiple varieties per class), both the from-scratch linear SVM and the from-scratch kernel (RBF) SVM reached 100% train/val/test accuracy, and 100% on a disjoint-capture-session hidden-test simulation -- a genuine result explained by Fruits-360's clean studio conditions rather than by the two models being equally good in general. A companion run of the identical codebase on a harder, more realistic synthetic dataset (Section 5) demonstrates the kernel trick's actual value: 74.7% (linear) vs. 92.0% (kernel) test accuracy, with a smaller train-to-test generalization gap for the kernel model. All code adheres to the assignment's Technical Requirements: numpy for all math, OpenCV/PIL only for image I/O and classical feature extraction, and no pre-built SVM, multiclass wrapper, or deep-learning classifier anywhere in the pipeline.